



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2

2024/2025

SECR2043-03 OPERATING SYSTEM

TITLE:

MEMORY MANAGEMENT – PAGING CONCEPT USING NETFLIX SCENARIO

NAME	MATRIC NUMBER
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

LECTURER: DR. DANLAMI GABI

Abstract

Memory management is vital in operating systems to ensure smooth program execution and resource efficiency. This project simulates logical-to-physical address translation using the paging technique, inspired by real-world systems, Netflix. Implemented in C, the simulation allows users to input logical addresses and observe step-by-step translation through a static page table. By demonstrating how pages are mapped to frames, this project helps users understand the fundamentals of paging and its role in preventing fragmentation, offering a practical learning tool for students studying operating systems.

Table of Content

1.0 Introduction.....	4
2.0 Problem Statement.....	4
3.0 Related Works.....	5
4.0 Methodology.....	6
4.1 Project Management Flowchart.....	6
4.2 Hardware and Software Requirements.....	7
4.3 Task Distribution.....	7
5.0 Design & Implementation.....	8
5.1 System Flowchart.....	8
Steps in Flowchart.....	8
6.0 Results & Discussion.....	12
6.1 Interface and Execution.....	12
6.2 Output Analysis.....	13
6.3 Discussion.....	14
7.0 Future Work & Conclusion.....	15
7.1 Future Work.....	15
7.2 Conclusion.....	16
Acknowledgment.....	17
References.....	18
Appendix.....	18

1.0 Introduction

Video streaming networks, for example, require efficient memory management to handle massive amounts of dynamic data. Paging is one of popular memory management techniques in operating systems that enable programs to access memory in fixed-size chunks without needing contiguous allocation. This project simulates how data pages are loaded and updates in memory as users interact with material, similar to how Netflix works. Moreover, the project demonstrates how operating systems manage memory efficiently to provide smooth performance and a positive user experience by imitating Netflix's data access utilizing paging.

2.0 Problem Statement

Systems such as Netflix must manage a large number of data pages in memory while remaining fast and efficient with the increased demand for high-quality video streaming and personalized content distribution. Without proper paging techniques, memory might become fragmented or overused, causing lags, buffering and system crashes. Many people are unaware of how memory is managed in the background to provide seamless streaming experiences. The goal of this project is to simulate paging using a Netflix-like interface to demonstrate how operating systems handle memory requests and page replacement, guaranteeing optimal resource utilization and user happiness.

3.0 Related Works

Paging is a common memory management method used by operating systems to support noncontiguous memory allocations, maximizing the use of RAM. Many educational tools and academic projects have replicated the concept of paging to show how logical addresses are associated with physical addresses and use a page table. For instance, animations from educational materials such as GeekforGeeks and simulations from textbooks provide resources to visualize the address translation process and are viewed through similar visual diagrams that illustrate the mechanisms.

In past research, paging has been used in simulations to allow the study of modern operating systems memory management. The projects simulated potential workload pressures that operating systems are designed to address (multitasking or other workloads). For training students or new developers or software engineers, these projects focused on the process of breaking a logical address into a page number and subsequent offset, which is used to look up a physical address in the page table and ultimately the translation process. Sadly, most of the tools are generally tactical and do not allow users to interact in real-time.

Using previous works as a foundation, this project adds interactive simulation elements. This project borrows some ideas from Netflix, where we need to use high-speed access to memory in order to accomplish synchronous streaming video. Users can enter logical addresses and see the procedures associated with the translation displayed in real-time. Accordingly, the students will have engaged in a form of active learning that improves important conceptual knowledge and demonstrates the importance of paging and how it maintains good performance and memory use in the contemporary systems.

4.0 Methodology

4.1 Project Management Flowchart

This project was executed through a structured development process as shown in Figure 4.1 :

1. Planning : Defined objectives, scope and simulation features
2. Research : Studied paging concepts and address translation mechanisms
3. Design : Developed system architecture and flowchart to stimulate address translation
4. Implementation : Wrote the simulation code in C Language
5. Testing : Validated address translation logic using simple input cases
6. Analysis and Reporting : Analyzed output and documented findings in this report

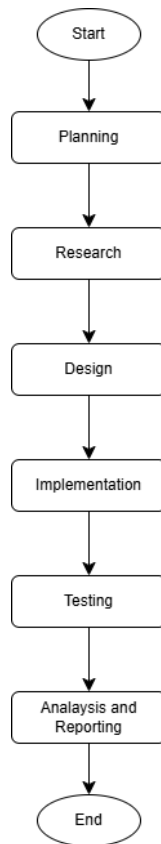


Figure 4.1 : Project Management Flowchart

4.2 Hardware and Software Requirements

This section outlines the necessary tools and platforms used to develop and execute the paging simulation program. Both hardware and software components are essential to ensure smooth execution, accurate testing and user-friendly interface for address translation simulation. The hardware and software used are as follows:

Hardware : Standard laptop or PC with C compiler

Software : C Compiler, Code Editor (VS Code), and Terminal/Command Prompt for program execution

4.3 Task Distribution

Tasks were distributed equally among group members to ensure efficient collaboration and timely completion of the project. This structured distribution allowed the team to work concurrently and maintain a consistent workflow throughout the project duration. Table 4.1 shows the task distribution for the group members.

Members	Tasks
Dayang Farah Farzana binti Abang Idham	● Flowchart & Simulation Design ● Writing report ● Video presentation
Farra Nurzahin binti Zaharil Anuar	● Flowchart & Simulation Design ● Writing report ● Video presentation
Nurul Adriana binti Kamal Jefri	● Literature review ● Writing report ● Video presentation
Safiya Nursyahadah binti Masnoor	● Coding & Testing ● Writing report ● Video presentation

Table 4.1 : Task Distribution

5.0 Design & Implementation

5.1 System Flowchart

Steps in Flowchart

1. Start Simulation

Prompt users to enter a logical address for translation.

2. Check Address Validity

Ensure the input logical address is within range of the defined logical memory which form 0 to 4095.

3. Calculate Page Number & Offset

Use $\text{logicalAddress} / \text{PAGE_SIZE}$ to determine the page number, and $\text{logicalAddress} \% \text{PAGE_SIZE}$ for the offset.

4. Look Up Frame in Page Table

Use the page number to retrieve the frame number from the page table.

5. Compute Physical Address

Compute the physical address using the formula :

$\text{physical Address} = (\text{frameNumber} * \text{PAGE_SIZE}) + \text{offset}$

6. Display the Result

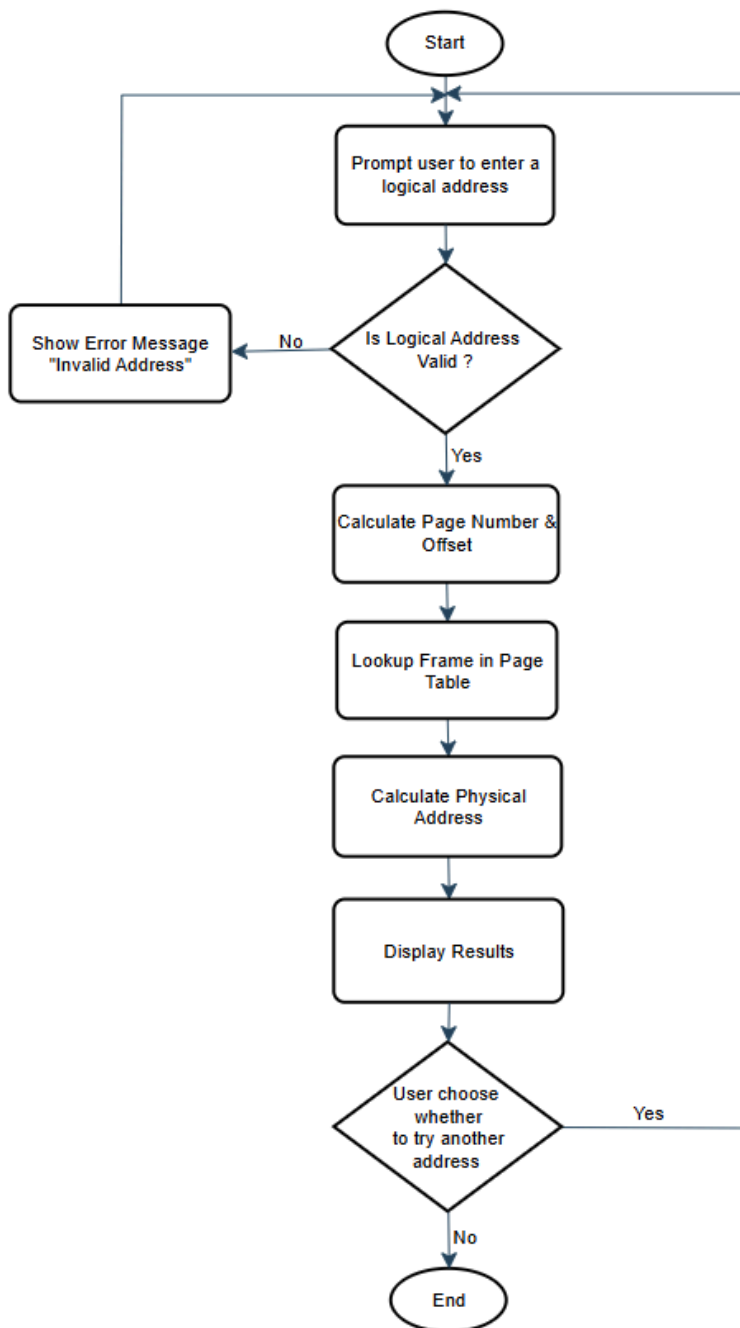
Output the page number, offset, frame number, and the final physical address.

7. Ask to Repeat

Ask the user whether to repeat the simulation with a new logical address.

If Yes : Go to Step 2

If No : End Simulation.



5.2 Code Implementation

Initialize Paging Simulation

Define constants for the page size, number of pages, and number of physical frames. Also define the page table which maps each logical page to a physical frame.

```
#define PAGE_SIZE 1024
#define NUM_PAGES 4
#define NUM_FRAMES 8
```

Prompt Logical Address

Prompt users to input a logical address. Input validation ensures the address falls within the logical memory range.

```
printf("\nEnter a logical address (0 - %d): ", PAGE_SIZE * NUM_PAGES - 1);
scanf("%d", &logicalAddress);
```

Address Translation Logic

Use integer division and modulus to calculate the page number and offset. Then use the page table to map the page number to a physical frame.

```
pageNumber = logicalAddress / PAGE_SIZE;
offset = logicalAddress % PAGE_SIZE;
physicalAddress = (pageTable[pageNumber] * PAGE_SIZE) + offset;
```

Display Translation Output

Print breakdown of the translation which are page number, offset, frame number, and physical address.

```
printf("Logical Address %d → Page %d + Offset %d\n", logicalAddress, pageNumber, offset);
printf("Mapped to Frame: %d\n", pageTable[pageNumber]);
printf("Physical Address = %d\n", physicalAddress);
```

Repeat Until User Quits

Use a do while loop to allow repeated address translation until the user chooses to exit.

```
printf("\nDo you want to enter another logical address? (y/n): ");
scanf(" %c", &again);
```

Complete Code

```
1  #include <stdio.h>
2  #define PAGE_SIZE 1024
3  #define NUM_PAGES 4
4  #define NUM_FRAMES 8
5
6  int main() {
7      int pageTable[NUM_PAGES] = {3, 1, 7, 5};
8      int logicalAddress, pageNumber, offset, physicalAddress;
9      char again;
10
11     printf("Paging Simulation (Logical to Physical Address Translation)\n");
12     printf("Page Size = %d bytes\n", PAGE_SIZE);
13     printf("Page Table:\n");
14     for (int i = 0; i < NUM_PAGES; i++) {
15         printf("Page %d --> Frame %d\n", i, pageTable[i]);
16     }
17
18     do {
19         printf("\nEnter a logical address (0 - %d): ", PAGE_SIZE * NUM_PAGES - 1);
20         scanf("%d", &logicalAddress);
21
22         if (logicalAddress < 0 || logicalAddress >= PAGE_SIZE * NUM_PAGES) {
23             printf("Invalid logical address.\n");
24         } else {
25             pageNumber = logicalAddress / PAGE_SIZE;
26             offset = logicalAddress % PAGE_SIZE;
27             physicalAddress = (pageTable[pageNumber] * PAGE_SIZE) + offset;
28
29             printf("Logical Address %d → Page %d + Offset %d\n", logicalAddress, pageNumber, offset);
30             printf("Mapped to Frame: %d\n", pageTable[pageNumber]);
31             printf("Physical Address = %d\n", physicalAddress);
32         }
33
34         printf("\nDo you want to enter another logical address? (y/n): ");
35         scanf(" %c", &again);
36     } while (again == 'y' || again == 'Y');
37
38     printf("\nSimulation ended.\n");
39     return 0;
40 }
```

6.0 Results & Discussion

6.1 Interface and Execution

The interface of the simulation is a command-line program written in C, which allows users to simulate how an operating system translates a logical address to a physical address using paging. The page table is hardcoded into the program for simplicity, consisting of 4 pages mapped to 8 available frames.

The simulation is executed by compiling the source code using a C compiler such as `gcc` and running the generated binary. The user is repeatedly prompted to enter a logical address between 0 and 4095 (because $4 \text{ pages} \times 1024 \text{ bytes/page} = 4096 \text{ bytes of logical address space}$).

Upon entering a logical address, the program:

- Calculates the corresponding page number and offset.
- Retrieve the frame number from the page table.
- Calculates and displays the physical address.

Example Execution:

```
Paging Simulation (Logical to Physical Address Translation)
Page Size = 1024 bytes
Page Table:
Page 0 --> Frame 3
Page 1 --> Frame 1
Page 2 --> Frame 7
Page 3 --> Frame 5

Enter a logical address (0 - 4095): 3072
Logical Address 3072 → Page 3 + Offset 0
Mapped to Frame: 5
Physical Address = 5120

Do you want to enter another logical address? (y/n): y

Enter a logical address (0 - 4095): 1500
Logical Address 1500 → Page 1 + Offset 476
Mapped to Frame: 1
Physical Address = 1500
```

This simple input-output interaction demonstrates the memory mapping process clearly. The simulation loop until the user input 'n', providing a chance to observe various mappings in a single session.

6.2 Output Analysis

The output analysis focuses on the translation steps and the correctness of the simulation in performing real OS memory management logic. Here's how the translation process works:

- **Page Number Calculation:**

Given a logical address of 3072 and a page size of 1024,

$$\text{Page Number} = 3072 / 1024 = 3$$

- **Offset Calculation:**

$$\text{Offset} = 3072 \% 1024 = 0$$

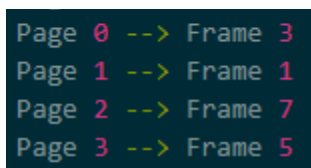
- **Frame Number Retrieval:**

Using `pageTable[3] = 5`, this maps to frame 5.

- **Physical Address Calculation:**

$$\text{Physical Address} = (5 \times 1024) + 0 = 5120$$

This page table used:



Page 0	-->	Frame 3
Page 1	-->	Frame 1
Page 2	-->	Frame 7
Page 3	-->	Frame 5

This fixed table allows easy observation of address translation behaviour across different pages. For instances:

- Logical address 0 → Page 0, Offset 0 → Physical Address 3072
- Logical address 4095 → page 3, Offset 1023 → Physical Address 6143

By examining a range of logical addresses, we can verify the consistent and accurate translation logic of the system.

6.3 Discussion

This paging simulation effectively demonstrates the memory management approach used in modern operating systems. Key concepts showcased include:

- **Page and Frame Abstraction:**

Logical memory is divided into pages; physical memory into frames. This allows memory to be managed without requiring contiguous blocks, thus minimizing external fragmentation.

- **Static Page Table Translation:**

The simulation uses a simple static page table, which represents how an OS looks up a frame number for a given page number.

- **Offset Preservation:**

The offset part of the logical address remains unchanged, demonstrating how only the page number needs translation.

- **Manual and Repeatable Testing:**

The interactive loop allows users to experiment with different logical addresses, promoting better understanding through trial-and-error.

However, this simulation also shows limitations of static and simplified models:

- No support for page faults or invalid page table entries.
- No dynamic page loading or swapping.
- Limited to a single process environment with a fixed address space.

Despite its simplicity, the simulation successfully captures the fundamentals of paging and serves as a strong educational tool.

7.0 Future Work & Conclusion

7.1 Future Work

While the current project accomplishes its goal of simulating logical-to-physical address translation through paging, there is significant potential for enhancements. Future improvements may include:

1. **Dynamic Page Table Input:**

Allow users to input their own page table mappings at runtime instead of using hardcoded values. This would allow testing of more diverse scenarios and promote better flexibility.

2. **Invalid Page Handling / Page Fault Simulation:**

Introduce simulation of page faults when a page is not present in the frame, along with page replacement mechanisms.

3. **Graphical User Interface (GUI):**

Develop a GUI using tools like **C++ with Qt** or **Python with Tkinter** to visually show the mapping of pages to frames, as well as a graphical memory layout.

4. **Extendable to Virtual Memory:**

Enhance the program to simulate **virtual memory** concepts like:

- Page swapping between RAM and disk.
- Page replacement algorithms (e.g., FIFO, LRU).

5. **Translation Lookaside Buffer (TLB):**

Implement a small TLB cache to simulate how modern systems speed up the translation process.

6. **Multi-process Simulation:**

Extend the simulation to include multiple processes with their own page tables to better reflect multitasking environments.

7.2 Conclusion

This project has successfully fulfilled its objective of simulating logical-to-physical address translation using paging, one of the core mechanisms in operating systems. The program allows users to input logical addresses and observe how they get translated into physical addresses via a page table.

This simulation allows users to really dive into how paging operates in a practical way. It covers everything from the memory division to the function of page tables and how address calculations work. This is an immensely useful tool for learning key memory management concepts in operating systems.

Although easy in method, the simulation provides a solid foundation for learning more complex topics with deeper understanding such as virtual memory, Lookaside Buffers (TLBs), and multiple-process address space management. With the proposed future developments in place, this project may be used as a solid instructional tool that graphically and interactively conveys complex OS memory behaviors.

Acknowledgment

We would like to extend our sincere appreciation to Dr. Danlami Gabi, our SECR2043 Operating System lecturer, for his references and guidance for the development of this project. His teachings have greatly enhanced our understanding of core memory management concepts, especially paging. We are also grateful to our group members, Dayang, Farra, Adriana and Safiya for their teamwork, dedication and strong collaboration in completing this project successfully. This project has been a valuable learning experience and we are thankful for the opportunity to apply theoretical knowledge to a practical simulation that enhances our understanding of how logical addresses are translated into physical addresses in a system. It has also given us hands-on experience in simulating a key memory management concept using C programming and has improved our problem-solving and technical writing skills. Overall, this was a valuable learning experience that contributed meaningfully to our development as computing students.

References

Jain, S. (2025, May 22). *Paging in Operating System*. GeeksforGeeks. Retrieved July 5, 2025, from <https://www.geeksforgeeks.org/operating-systems/paging-in-operating-system/>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating system concepts* (9th ed.). John Wiley & Sons.

GeeksforGeeks. (2025, May 22). *Paging in the operating system*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/paging-in-operating-system/>

Appendix

Youtube Link : <https://youtu.be/jwJKF3Vg3t8>